THE DATA ENGINEER LAB

# DATABRICKS FROM ZERO TO HERO

*A 30-Day Interview & Career Readiness Series*

## DAY 8

**ACID Transactions in Delta Lake — How It Works Without Locks**

SECTION 01

# Quick Recap — What ACID Means

| Guarantee | What It Means for a Delta Table |
|---|---|
| Atomicity | A write either fully commits or doesn't happen at all; no partial writes. |
| Consistency | The table always reflects a valid, complete state defined by the transaction log. |
| Isolation | Concurrent transactions don't interfere with or corrupt each other. |
| Durability | Once a transaction commits, it survives crashes, restarts, and outages. |

Day 7 covered what these guarantees are. This document goes one level deeper: the actual mechanism, called optimistic concurrency control, that lets Delta Lake deliver all four guarantees without traditional database-style locking.

SECTION 02

# Optimistic Concurrency Control (OCC)

**CORE CONCEPT**

Delta Lake uses optimistic concurrency control to provide transactional guarantees between writes. Writes operate in three stages: reading the latest version of the table to identify which files need to be modified, writing the new data files, and then checking whether the proposed changes conflict with anything committed since that snapshot was read.

## The Three Stages

- Read: the writer reads the latest available version of the table to identify which files need modification. Append-only writes skip this step since they don't need to read current state.
- Write: the writer stages new data files in the directory used to define the table, without touching or locking existing files.
- Validate and Commit: before committing, Delta checks whether the proposed change conflicts with anything committed concurrently since the snapshot was read. If there's no conflict, the change is committed as a new versioned snapshot and the write succeeds.

**WHY 'OPTIMISTIC'**

Optimistic means Delta Lake assumes conflicts between writers are rare and doesn't lock resources upfront. Each writer proceeds independently, and conflicts are only detected at commit time, unlike pessimistic locking approaches used by traditional databases, which acquire locks before modifications and can block other writers even when no real conflict would have occurred.

SECTION 03

## What Happens When Two Writers Conflict

Consider two transactions modifying overlapping data in the same file at nearly the same time.

```
T1: DELETE row from FileA   -----> commits first, recorded in log
T2: DELETE another row      -----> checks log before committing
     from FileA                    finds conflict with T1
                                    T2 is rejected / retried

Result: FileA stays consistent. No corrupted or lost writes.
```

Transaction 1 finishes first and is recorded in the transaction log. Before Transaction 2 is recorded, Delta checks the metadata, finds that it conflicts with the already-recorded transaction, and rejects it, typically causing the operation to retry rather than silently corrupt the table.

SECTION 04

## Isolation Levels on Databricks

Isolation level determines exactly how strict the conflict-checking is between concurrent operations. Databricks supports two isolation levels for Delta tables.

| Isolation Level | Behavior |
|---|---|
| WriteSerializable (default) | Provides strong consistency with better performance; slightly more permissive about what counts as a conflict. |
| Serializable | The strictest isolation level; guarantees transactions behave as if executed one at a time, in some serial order. |

**GOOD TO KNOW**

By default, Databricks provides snapshot isolation on reads and write-serializable isolation on writes. There are no locks on reading or writing against a table, which means deadlock is not possible, and readers always see a consistent snapshot of the table even if it's modified mid-query.

SECTION 05

## Optimistic Concurrency vs Pessimistic Locking

| Aspect | Pessimistic Locking (Traditional DB) | Optimistic Concurrency (Delta Lake) |
|---|---|---|
| When Conflicts Are | Upfront, before any writes happen | At commit time, after writes are |

| | | |
|---|---|---|
| Checked | | staged |
| Blocking Behavior | Other writers are blocked until lock is released | Writers proceed independently in parallel |
| Best Suited For | Workloads with frequent, high-conflict writes | Append-heavy workloads with rare conflicts |
| Failure Mode | Writers wait or deadlock | Conflicting writer retries the transaction |

SECTION 06

## Important Boundaries to Know

- Delta Lake manages transactions at the table level; transactions always apply to one table at a time by default.
- Multi-table, multi-statement transactions are only possible on tables with catalog commits enabled, and are not the default behavior.
- Primary key and foreign key relationships on Databricks are informational only; they are not enforced the way they would be in a traditional relational database.
- Concurrent writes can conflict and cause a retry, but Delta Lake is designed so they never corrupt the table itself.

SECTION 07

## Glossary — Day 8 Terms

| Term | Definition |
|---|---|
| Optimistic Concurrency Control (OCC) | A concurrency strategy that assumes conflicts are rare, allowing writers to proceed without locks and checking for conflicts only at commit time. |
| Pessimistic Locking | A concurrency strategy that acquires locks before modifications, blocking other writers even when no real conflict exists. |
| WriteSerializable | The default Databricks isolation level, balancing strong consistency with write performance. |
| Serializable | The strictest isolation level, guaranteeing transactions behave as if executed one at a time. |
| Snapshot Isolation | A guarantee that readers see a consistent view of the table as of when their query started, even if the table changes mid-query. |
| Conflict Retry | The process where a transaction that conflicts with a concurrently committed change is rejected and re-attempted. |

SECTION 08

# Interview Questions & Answers

**Q1. How does Delta Lake provide ACID guarantees without locking the table?**

**A.** It uses optimistic concurrency control, where writers read the latest table version, stage their writes independently, and only check for conflicts at commit time, rather than locking resources upfront.

**Q2. What are the three stages of a Delta Lake write under optimistic concurrency control?**

**A.** Read, where the latest table version is read to identify files needing modification; Write, where new data files are staged; and Validate and Commit, where the proposed change is checked against anything committed concurrently before being finalized.

**Q3. What happens when two writers conflict under optimistic concurrency control?**

**A.** The transaction that commits first is recorded in the transaction log. The second transaction is checked against the log, found to conflict, and is rejected, typically triggering a retry rather than corrupting the table.

**Q4. What is the difference between WriteSerializable and Serializable isolation?**

**A.** WriteSerializable is the default on Databricks, balancing strong consistency with better performance. Serializable is the strictest isolation level, guaranteeing transactions behave as if executed one at a time in some serial order.

**Q5. Why is optimistic concurrency control well suited to data lake workloads?**

**A.** Because most data lake writes are appends to different partitions or time ranges, true conflicts are rare, so the overhead of upfront locking is unnecessary; occasional retries are cheaper than constant distributed locking.

**Q6. Are primary key and foreign key constraints enforced in Delta Lake?**

**A.** No. On Databricks, primary key and foreign key relationships are informational only and are not enforced the way they would be in a traditional relational database.

**STAY ON TRACK FOR ALL 30 DAYS**

Comment the keyword **DATABRICKS** on the reel to get this document and future deep-dives sent directly to you.

*Piyush Varma | The Data Engineer Lab*